

CS 217: Artificial Intelligence and Machine Learning Spring 2025-26, End-Semester exam

April 29, 2026.
5:30–8:30 pm

Roll: _____

Name: _____

Mode: Credit/Audit/Sit-through _____

Write all your answers in the space provided. Do not spend time/space giving irrelevant details or details not asked for. *But you need to show the important steps that are required to justify the answer.* Use the marks as a guideline for the amount of time you should spend on a question. You are only allowed to refer your hand-written notes, no one else's notes or textbook.

1. A hospital is trying to classify patients as **Diabetic** ($y = 1$) or **Healthy** ($y = 0$) based on two attributes: x_1 denoting fasting blood glucose level (in mmol/L, rounded to the nearest integer) and x_2 denoting smoking status (N = non-smoker, S = smoker). The table below shows a sample training set.

	x_1	x_2	y
$\mathbf{x}^0$	4	N	0
$\mathbf{x}^1$	8	S	1
$\mathbf{x}^2$	5	N	0
$\mathbf{x}^3$	7	S	1
$\mathbf{x}^4$	6	N	0
$\mathbf{x}^5$	9	S	1
$\mathbf{x}^6$	6	S	1
$\mathbf{x}^7$	5	S	0

- (a) First, we will fit a simple Naive Bayes classifier on this dataset where $P(x_1|y) \sim \mathcal{N}(\mu_y, \sigma_y^2)$, $P(x_2|y) \sim \text{Bernoulli}(\theta_y)$. Assuming a Beta $B(\alpha, \beta)$ prior on θ_y .

Write down the Bayesian (MAP) estimates of the θ_y parameters, and maximum likelihood estimate of the μ_y, σ_y parameters for class $y = 1$ below [Show derivations as needed] ..3

[Rubric: (+1) mark for correct μ , (+1) mark for correct σ and (+1) mark for correct MAP estimate over θ]

The log-likelihood of x_1 for class $y = 1$ (patients $\mathbf{x}^1, \mathbf{x}^3, \mathbf{x}^5, \mathbf{x}^6$, with x_1 values 8, 7, 9, 6) is

$$\ell(\mu_1, \sigma_1^2) = -\frac{n_1}{2} \log(2\pi\sigma_1^2) - \frac{1}{2\sigma_1^2} \sum_{i:y^i=1} (x_1^i - \mu_1)^2.$$

Setting $\partial\ell/\partial\mu_1 = 0$ gives $\hat{\mu}_1 = \frac{1}{n_1} \sum_{i:y^i=1} x_1^i = \frac{8+7+9+6}{4} = \mathbf{7.5}$.

Setting $\partial\ell/\partial\sigma_1^2 = 0$ gives $\hat{\sigma}_1^2 = \frac{1}{n_1} \sum_{i:y^i=1} (x_1^i - \hat{\mu}_1)^2 = \frac{(0.5)^2 + (0.5)^2 + (1.5)^2 + (1.5)^2}{4} = \mathbf{1.25}$.

Class $y = 1$ ($n_1 = 4$; smokers: $\mathbf{x}^1, \mathbf{x}^3, \mathbf{x}^5, \mathbf{x}^6$ all have $x_2 = \text{S}$, so $n_{1S} = 4$, $n_{1N} = 0$). Since θ_y follows a Beta prior we get the MAP as

$$\hat{\theta}_1 = \frac{4 + \alpha}{4 + \alpha + \beta}$$

Since the question did not specify if 'S' is 0/1, we will grade the opposite interpretation too.

NOTE for Cribs:

- The question specifically asks MAP for class $y = 1$ only, any crib with showing answers for another class will not be entertained.
- For the final answer, suggestion from the students was considered and the evaluation has been done using the revised solution

$$\hat{\theta}_{MAP} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2}$$

So final correct answer can be:

$$\hat{\theta}_1 = \frac{\alpha + 3}{\alpha + \beta + 2}$$

OR

$$\hat{\theta}_1 = \frac{\alpha - 1}{\alpha + \beta + 2}$$

- (b) Now suppose in general we have N instances and we have already estimated the Bayesian parameters $\mu_y, \sigma_y, \theta_y$. What additional statistics over N will you keep around, so that you can efficiently add an $N + 1$ th instance to the training set and update the parameters. ..2

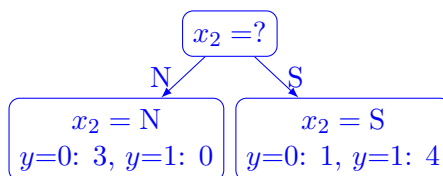
[**Rubric:** (+1) mark for correct mention of additional statistics for μ & σ and (+1) mark for correct mention of additional statistics for θ_y]

For μ, σ keep around sum of x_1 s, sum of squares of x_1 s, and count.

For θ_y keep total count and count of "S"s.

[**NOTE for Cribs:** Full marks have been awarded if you properly mention all the statistics given in the answer key, in case any student mentions that only count is required then full marks have been given if they mention the proper detail of how it will be done using count only.]

- (c) Suppose we build a decision tree and the first split is on x_2 . Draw the two branches and state the number of examples from each class in each branch. ..1



[**Rubric:** (+0.5) mark for correct number of examples for N and (+0.5) mark for correct number of examples for S]

The N branch contains $\mathbf{x}^0, \mathbf{x}^2, \mathbf{x}^4$ (all $y = 0$) and the S branch contains $\mathbf{x}^1, \mathbf{x}^3, \mathbf{x}^5, \mathbf{x}^6$ ($y = 1$) and $\mathbf{x}^7$ ($y = 0$).

- (d) What is the reduction in Gini due to the split in part (c)? ..2

[**Rubric:** (+0.5) mark for GINI before split N , (+0.5) GINI of the N-branch, (+0.5) GINI of the S-branch, and (+0.5) mark for GINI after split]

TODO: rework in terms of Gini.

Let H denote Gini.

Gini before the split (4 positives, 4 negatives out of 8):

$$H(\text{root}) = 1 - 2 * (1/2)^2$$

Gini of the N-branch (3 examples, all $y = 0$):

$$H(N) = 0.$$

Gini of the S-branch (5 examples, 1 of class 0, 4 of class 1):

$$H(S) = 1 - (1/5)^2 - (4/5)^2 = 25 - 17/25$$

Weighted Gini after split:

$$H_{\text{after}} = \frac{3}{8} \cdot 0 + \frac{5}{8} (8/25)$$

Gini reduction:

$$IG(x_2) = H(\text{root}) - H_{\text{after}}$$

[NOTE for Cribs: One thing to note some students have only mentioned GINI after split in that case, the marks have been given if the final value of GINI after split is correct.]

- (e) Now again suppose that in general we have N instances and we have already created a decision tree T_N over them. We seek to add an $N + 1$ th instance to the training set, and find the example is already correctly classified by T_N . Does it mean that if were to recreate the tree T_{N+1} from scratch over all $N + 1$ examples, we would get back T_N ? Justify your answer. No marks without proper justification. ..3

[Rubric: (+3) Only for correct answer and proper justification with building the example tree from scratch, before and after adding the data point. No, in general.

Here is an example. Assume 2 classes. Say we have a 1-D dataset with (x, y) pairs as $\{(1,0),(2,1),(3,0),(4,1),(5,0),(6,1)\}$ This split at $x \leq 1.5$ with Left: 1 point, all class 0 and Right: $\{2,3,4,5,6\}$ with 2 zeros, 3 ones with $1(2/5)^2(3/5)^2 = 0.48$ Weighted Gini: $(1/6)(0) + (5/6)(0.48) = 0.40$

Now, if we add a new point $(7,1)$, the best split can be seen to be at $x \leq 5.5$.

We have to make sure the example that students give starts with T_N that is good.

- (f) Consider a linear classifier using only x_1 $f(x) = w_0 + w_1x_1$ with decision rule $\hat{y} = \text{sign}(f(x))$.
- (i) Assume the loss function is $\max(0, -(2y - 1)f(x))$. [This is called the perceptron loss.] Assume we apply the stochastic subgradient descent algorithm to learn the parameters w_0, w_1 to minimize the overall loss on the training examples. Say, the initial value of $\mathbf{w}^t = (w_0^t, w_1^t) = (-10, 1)$ at $t = 0$. And we first visit a mini-batch with just a single example $\mathbf{x}^6 = (6, S)$ with $y^6 = 1$. Assume learning rate is 1. What is the updated value of parameters $\mathbf{w}^{t+1}$? ..2 **[Rubric:** (+0.5) Correctly evaluating $f(\mathbf{x}^6)$ and determining the example is misclassified. **[Rubric:** (+1) Correctly computing the subgradient $[-1, -6]$. **[Rubric:** (+0.5) Correctly applying the update with learning rate 1 to get $\mathbf{w}^{t+1} = (-9, 7)$

$f(\mathbf{x}^6)$ with $\mathbf{w} = (-10, 1)$ is $-10 + 1 \cdot 6 = -4 < 0$, so $\hat{y} = 0 \neq 1$: misclassified. The value of the loss for the $\mathbf{w}$ is then $-(2 * 1 - 1)(w_0 + w_1(6))$ and its subgradient is $[-1, -6]$ and we get the update

$$(w_0, w_1) \leftarrow (-10, 1) + (1)(1, 6) = (-9, 7).$$

- (ii) Now again suppose that in general we have N instances and we have already learned $\hat{\mathbf{w}} = (\hat{w}_0, \hat{w}_1)$. We seek to add an $N + 1$ th instance to the training set, and find the example is already correctly classified by $\hat{\mathbf{w}}$. Does it mean that if we were to minimize the loss above from scratch we would converge to a lower loss over the $N + 1$ examples? Justify your answer. No marks without proper justification.

..2 [Rubric: (+1) for correct answer and if some justification is made. [Rubric: (+1) When the justification is correct No, the loss will be equal, not lower.

The perceptron loss is convex (max of linear functions, summed), so subgradient descent on N examples converges to a global minimum $L_N^* = L_N(\hat{w})$. Since the new $(N + 1)$ -th example is correctly classified by $\hat{w}$, it contributes zero perceptron loss, giving $L_{N+1}(\hat{w}) = L_N^* + 0 = L_N^*$.

Hence $L_{N+1}^* \leq L_N(\hat{w}) = L_N^*$. Conversely, since every per-example loss is non-negative, $L_{N+1}(w) \geq L_N(w)$ for all w , so $L_{N+1}^* \geq L_N^*$. Combining, $L_{N+1}^* = L_N^*$: retraining from scratch achieves exactly the same minimum loss, neither lower nor higher.

- (g) Consider training a linear SVM using only x_1 as a feature. Assume $\mathbf{x}^6$ is dropped from the training set for this question.

- (i) Identify the support vectors. ..1 If $\mathbf{x}^6$ is removed from the dataset, the support vectors are $\mathbf{x}^4$ ($x_1 = 6, y = 0$) and $\mathbf{x}^3$ ($x_1 = 7, y = 1$).

[Rubric: (+0.5) for identifying each support vectors $\mathbf{x}^i(x_1, y)$.]

- (ii) Write the equation of the maximum-margin decision boundary assuming that $\xi_i = 0$ for all training instances. ..2 With support vectors at $x_1 = 6$ ($y = 0$) and $x_1 = 7$ ($y = 1$), the SVM places the boundary at the midpoint:

$$x_1 = 6.5.$$

For the support vectors we want $w_1 x_1 + b = 1$ and for the midpoint we want $w_1 x_1 + b = 0$. This gives $w_1 = 2, b = -13$

This solution $w_1 = 1, b = -6.5$ is incorrect since it does not keep the support vectors at distance 1 [Rubric: (+1) for identifying the boundary location, (+1) for writing the final equation of the decision boundary]

- (iii) Now again suppose that in general we have N instances, we are training an SVM for a Kernel $K(\mathbf{x}, \mathbf{x}')$ in the dual space using the SMO algorithm and we have already learned the optimal dual parameters (α_i) s. We get an $N + 1$ th instance to the training set. Suggest how will you update the parameters with the new example, and identify if there is any case where no updates will be required. ..3 We calculate if for the new point $\xi_{N+1} = 0$. If yes, the trained SVM is optimal. Else, we run the SMO algorithm starting with X_{N+1} as one instance, and choose another one to pair based on the SMO rule. [Rubric: (+2) for approach to update parameters with new example, (+1) for identifying case where 'no updates will be required'.]

2. Last week in the news, we saw humanoid robots compete with humans in a long-distance running race conducted in China. Some of the humanoid robots successfully completed the race.

Suppose you intend to train a humanoid robot for locomotion (either walking or running) using reinforcement learning. How would you formulate a corresponding MDP? In particular, specify your definitions of states, actions, and rewards. What would be the transition function?

..3 Humanoid robots have multiple joints (usually about 6 in each leg and 4 in each arm), whose angle is controlled by specifying torques. The state of the robot is defined by the positions and velocities of the joints, and additionally, the position and velocity of some reference point (an “origin” in the robot). The latter is usually recorded using an inertial measurement unit.

Each action is a vector that gives the torque to control each joint. Sometimes, low-level controllers take care of specifying the torques, and higher level actions could be, for example, “lean to the left” or “take a 40cm stride”. Depending on the level of abstraction of the control, the frequency of action-taking can vary from 20-50 times a second, to once every few seconds.

To train a bipedal robot to move, the major reward must be for forward velocity (or distance covered if time is held constant). Training can also benefit from penalties for falling.

The transitions come from physics!—either in the real world or on a simulator. Given state and action, the laws of physics (gravity, friction, etc.) are applied to determine the next state.

[**Rubric:** (+1) for defining a state space with joint positions/velocities and base pose and actions as torques or high-level commands; (+1) for a reward function focused on forward velocity and fall prevention; (+1) for identifying transitions as being governed by the laws of physics.]

3. You are given N points $\mathbf{x}_1, \dots, \mathbf{x}_N$, a distance function $d(\mathbf{x}, \mathbf{x}')$ that is a metric, and k a desired number of clusters. Your goal is to assign to each point $\mathbf{x}_i$ a number from 1 to k such that the pairwise distance among all points with the same cluster id is minimized. In class, we discussed the k -means algorithm for solving this task. Now, we will try to solve it via the A^* search algorithm. Here there is no explicit representation of the cluster centers, only assignment of points to cluster ids. Let $C(\mathbf{x}_i) \mapsto [1 \dots k]$ denotes cluster id of point $\mathbf{x}_i$ which we need to search via A^* . Assuming that initially all $C(\mathbf{x}_i)$ are un-assigned, and the state n is defined by assignments of $C(\mathbf{x}_i)$ to possibly a subset of the points. An action consists of assigning a cluster id to an unassigned point. A point once assigned a cluster-id cannot be unassigned. A goal state is reached when all N $C(\mathbf{x}_i)$ are assigned. For example, when $N = 4, k = 2, C(\mathbf{x}_1) = 1, C(\mathbf{x}_2) = 1, C(\mathbf{x}_3) = 2, C(\mathbf{x}_4) = 1$, we have reached a goal state and the cost is $d_{12} + d_{24} + d_{14}$ where $d_{ij} = d(\mathbf{x}_i, \mathbf{x}_j)$.

- (a) List the cost function $g(n)$ for a state as per our defined goal. ..1

Let S_n be the set of indices of points assigned to a cluster in state n . The cost function is the sum of pairwise distances between all points in S_n that share the same cluster ID:

$$g(n) = \sum_{\{i,j\} \subseteq S_n, i < j} \mathbb{I}(C(x_i) = C(x_j)) \cdot d(x_i, x_j)$$

Where:

- $\mathbb{I}(\cdot)$ is the indicator function (1 if the condition is true, 0 otherwise).
- $C(x_i)$ is the cluster ID assigned to point x_i .

[**Rubric:** (+1) for correctly defining $g(n)$ as the sum of pairwise distances between points assigned to the same cluster index in the current state.]

- (b) Design a consistent heuristic $h(n)$ for this task. [Marks will depend on the tightness of $h(n)$ vis-a-vis the true cost. No marks if your heuristics is trivial, and/or inconsistent.]

..4 Many possible. Here is one.

- i. In any goal state, the total number of internal edges (pairs in the same cluster) is $\sum_{c=1}^k \binom{n_c}{2}$, where n_c is the number of points in cluster c . This sum is minimized when the clusters are as balanced as possible (i.e., each cluster has roughly N/k points). Let U be this minimum number of edges:

$$U = \sum_{c=1}^k \frac{n_c(n_c - 1)}{2} \text{ where } \sum n_c = N$$

- ii. Let r be the number of edges already accounted for in $g(n)$ (pairs of assigned points with the same ID).
- iii. To reach a valid goal state, we must eventually "commit" at least $U - r$ more internal edges.
- iv. Let E_{open} be the set of all pairs (x_i, x_j) that are not yet "committed" to being in the same cluster (specifically, pairs where at least one point is currently unassigned). To ensure the heuristic is a lower bound, we sum the $U - r$ smallest distances available in E_{open} :

$$h(n) = \sum \text{smallest } (U - r) \text{ distances from } E_{open}$$

4. We are given a dataset $\{\mathbf{x}_n\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^d$ for each of which we want low dimensional embedding $\mathbf{z}_n \in \mathbb{R}^L$, $L < D$. We studied two methods:

PCA: that encode and decode with orthogonal matrix $W \in \mathbb{R}^{d \times L}$:

$$\mathbf{z} = W^\top \mathbf{x} \quad (\text{encode}), \quad \hat{\mathbf{x}} = W \mathbf{z} \quad (\text{decode})$$

Auto-Encoder (AE) that trains two networks: an encoder $\mathbf{z} = f_e(\mathbf{x})$ and a decoder $\hat{\mathbf{x}} = f_d(\mathbf{z})$ such that $\hat{\mathbf{x}} = f_d(f_e(\mathbf{x}; \theta_e); \theta_d)$

In both cases the goal is to minimize the loss: $\mathcal{L}(\theta) = \sum_n \|\hat{\mathbf{x}}_n - \mathbf{x}_n\|^2$

Suppose we learn $f_e(\mathbf{x}) = U^\top \mathbf{x}$ and $f_d(\mathbf{z}) = V \mathbf{z}$ where $U, V \in \mathbb{R}^{d \times L}$. Assume the training examples are centered so that their mean is 0.

- (a) Assuming the AE achieves a globally minimum loss objective, are the learned optimal parameters of the AE guaranteed to agree with PCA's solution? Justify your answer.

..2 [**Rubric:** (+2)] For solution saying 'NO' with correct justification.

[**Rubric:** (+1)] For solution saying 'NO' with vague or incomplete justification.

No, not in general.

PCA constraints U and V to be the same, and forces W to be orthogonal. No such guarantees in the AE.

- (b) Consider another Auto-encoder where the input is an integer from 1 to d represented as a one-hot vector. For example, when input integer is 2, and $d = 7$, then it is provided to the AE as $\mathbf{x} = [0, 1, 0, 0, 0, 0, 0]'$. You wish to represent it as a code $\mathbf{z}$ of dimensionality L where $d = 2^L$. Provide one optimal design of f_e, f_d where f_e and f_d are feed-forward networks. [Hint: a linear layer with a ReLU should suffice.] ..4

[Rubric: (+1.5)] For encoding design.

[Rubric: (+1.5)] For decoding design.

[Rubric: (+0.5)] Architecture validity.

[Rubric: (+0.5)] Output correctness.

$\mathbf{z}$ is binary encoding of the integer that $\mathbf{x}$ represents. For that the f_e can be a single linear layer whose parameters U just stores the binary code of each integer i in each row of U with the only difference that we just -1 instead of 0. Then, the V is just the transpose of the U , and $b = -L + 1$ for all positions followed by a ReLU.

Many other solutions possible.

5. Consider the method for learning distance functions using Siamese networks that attempts to learn $G_W(\mathbf{x})$ to minimize the following loss function on sum of all pairs of instances (i, j) created from a labeled classification dataset D :

$$L(W, i, j) = \delta(y_i = y_j) d(\mathbf{x}^i, \mathbf{x}^j) + \delta(y_i \neq y_j) \max(0, m - d(\mathbf{x}^i, \mathbf{x}^j)) \quad (1)$$

where $d(\mathbf{x}^i, \mathbf{x}^j) = \|G_W(\mathbf{x}^i) - G_W(\mathbf{x}^j)\|^2$ and $\delta(c)$ is 1 when the condition c is true, and 0 otherwise, and $m > 0$ is a margin hyper-parameter. Consider the case where G_W comprises of only a linear layer so that $G_W(\mathbf{x}) = W\mathbf{x}$ where W is a $d \times d$ tensor, and $\mathbf{x}$ is a d -dimensional vector.

- (a) What is the value of the total loss when applied on a batch of 10 instances, two classes, and equal number of examples in each class, when $W = 0$. ..1 We have $5 \times 4 / 2$ distinct pairs of the same class, and 5×5 pairs of opposite classes. At $W = 0$ all distances are zero. The loss of same class pairs is 0, and opposite class pairs is m . Thus total loss is $25m$. RUBRIC: +1 for correct total loss. Also considering $50m$ as correct.
- (b) What is the gradient with respect to parameter W_{rs} of the above loss function at $W = I$ where I denotes a $d \times d$ identity matrix with zeros in the off-diagonal entries and 1s in the diagonal entries. [Hint: Apply the backprop algorithm.] ..3

$$\begin{aligned} \frac{\partial L}{\partial w_{jk}} &= \sum_i \sum_{j=i+1}^N \frac{\partial L(w, \hat{x}_j)}{\partial d(\mathbf{x}^i, \mathbf{x}^j)} \cdot \frac{\partial}{\partial w_{rs}} \left(\sum_{j'} \left(\sum_{k'} w_{j'k'} x_{k'}^i - \sum_{k'} w_{j'k'} x_{k'}^j \right)^2 \right) \\ &= 2 \left(\sum_{k'} w_{\sigma k'} x_{k'}^i - \sum_{k'} w_{\sigma k'} w_{rk'} x_{k'}^j \right) [x_s^i - x_s^j] \Big|_{w=I} \\ &= 2 (\hat{x}_s^i - \hat{x}_s^j) (x_s^i - x_s^j) \end{aligned}$$

RUBRIC: +2 for correct expression via chain rule. +1 for correct evaluation at $W = I$, leading to the correct final expression.

- (c) What would go wrong if we instead use the objective without margin? That is, with the loss objective:

$$L(W, i, j) = \delta(y_i = y_j) d(\mathbf{x}^i, \mathbf{x}^j) - \delta(y_i \neq y_j) d(\mathbf{x}^i, \mathbf{x}^j) \quad (2)$$

Consider here the general case where G_W is a universal neural network. .2

The objective can get unbounded if there are coordinates where instances of the same class agree, and different classes disagree. A neural network training can seek such G_W causing the objective to keep reducing without actually learning any meaningful distance function to bring together examples of the same class. RUBRIC: +1 for identifying that the loss becomes unbounded, and under what conditions it occurs.

6. Suppose you are charged with training a classifier for a company that categorizes every Tweet about the company into whether it expresses a positive or negative sentiment. You managed to painfully assign labels via manual inspecting to only 100 tweets, but you managed to download 10,000 tweets that mention the name of the company. State what model you will use to classify tweets and what tricks you can use to improve the accuracy of your sentiment classifier. Justify your answer. [Limit your answer to selecting from models and architectures covered in class.] .3

We will start with the BERT pre-trained model, and fine-tune it on the 10,000 unlabeled tweets of the company with masked reconstruction loss. This teaches the model meaningful representation of the words associated with the company's tweets. Now on each labeled sentence we use the CLS token embedding from BERT as an input vector to a small feedforward network (could be a single linear layer) whose parameters are trained using the limited labeled data. RUBRIC: +1 for correct choice of model AND reasoning. +2 for identifying how to tackle the data labeling bottleneck.

7. Given an input sequence $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^T$, where $\mathbf{x}_i \in \mathbb{R}^d$, recall that the standard scaled dot-product attention is defined as:

$$\mathbf{O} = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} \right) \mathbf{V} \quad (3)$$

where $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_N]^T = \mathbf{X}W^Q$, $\mathbf{K} = \mathbf{X}W^K$, and $\mathbf{V} = \mathbf{X}W^V$ represent the query, key, and value sequences, and $W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$ denote the learned weight matrices.

Now, let us propose **Clustered Attention** as an approximation to the standard attention mechanism described in Eq 3. At a high level, the idea of clustered attention is to replace every vector $\mathbf{q}_i \in \mathbf{Q}$ and $\mathbf{k}_j \in \mathbf{K}$ with their nearest corresponding centroids, prior to performing attention computations.

More formally, we define two approximation functions $\Phi^q : \mathbb{R}^d \rightarrow \mathcal{C}^Q$ and $\Phi^k : \mathbb{R}^d \rightarrow \mathcal{C}^K$, where $\mathcal{C}^Q = \{\mathbf{c}_1^Q, \mathbf{c}_2^Q, \dots, \mathbf{c}_M^Q\}$ and $\mathcal{C}^K = \{\mathbf{c}_1^K, \mathbf{c}_2^K, \dots, \mathbf{c}_M^K\}$ are learned sets of M cluster centroids corresponding to the query and key spaces, respectively (with M being a constant defined during model creation such that $M \ll N$). For the time-being, you can assume that these centroids are provided to you. More details about how $\mathcal{C}^Q$ and $\mathcal{C}^K$ are obtained is outlined in part (c). Now, for any vector $\mathbf{a} \in \mathbb{R}^d$, these functions are defined as:

$$\Phi^q(\mathbf{a}) = \text{argmin}_{\mathbf{c} \in \mathcal{C}^Q} \|\mathbf{a} - \mathbf{c}\|_2 \quad \Phi^k(\mathbf{a}) = \text{argmin}_{\mathbf{c} \in \mathcal{C}^K} \|\mathbf{a} - \mathbf{c}\|_2 \quad (4)$$

In simpler terms, Φ^q and Φ^k map every d -dimensional vector to its nearest neighbour in $\mathcal{C}^Q$ and $\mathcal{C}^K$ respectively. Utilizing these functions, we can now define **Clustered Attention** as:

$$\mathbf{O}^c = \text{Softmax} \left(\frac{\Phi^Q(\mathbf{Q})(\Phi^K(\mathbf{K}))^T}{\sqrt{d}} \right) \mathbf{V} \quad (5)$$

where $\Phi^Q(\mathbf{Q}) = [\Phi^q(\mathbf{q}_1), \Phi^q(\mathbf{q}_2), \dots, \Phi^q(\mathbf{q}_N)]^T$ and $\Phi^K(\mathbf{K}) = [\Phi^k(\mathbf{k}_1), \Phi^k(\mathbf{k}_2), \dots, \Phi^k(\mathbf{k}_N)]^T$.

- (a) If we have a degenerate case where all centroids in $\mathcal{C}^Q$ and $\mathcal{C}^K$ end up being same (i.e., $\mathbf{c}_i^Q = \mathbf{c}_j^Q = \mathbf{c}_i^K = \mathbf{c}_j^K = \mathbf{c} \forall i, j \in \{1, \dots, M\}$), and such centroids are used in Eq 5, what would happen to the output sequence $\mathbf{O}^c$? Justify.

..1 All output vectors would be identical. Since all the cluster centroids are the same, regardless of how the clusters get assigned, the attention matrix will be uniform. Thus, every output vector would simply be the average of all value vectors $\frac{1}{N} \sum_{i=1}^N \mathbf{v}_i$.

Rubric: +1 for stating correct behaviour with justification.

- (b) During model deployment, you are informed that the dot-product operation $q_i \cdot k_j$, which is required for calculating the attention matrix, is computationally very expensive on the production server. Hence, you are allowed to use at most M^2 such operations per input sequence. Given $\mathbf{X}, \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathcal{C}^Q$, and $\mathcal{C}^K$, describe an efficient algorithm that computes $\mathbf{O}^c$ (Eq:5) within this computational budget. [Hint: The algorithm will take $O(M^2d + NMd)$ time complexity. No need to get into hardware-level optimization tricks. Focus on high-level algorithmic strategy.]

..3 The trick here is to realize that if two input vectors $\mathbf{q}_i$ and $\mathbf{q}_j$ get mapped to the same cluster centroid, then both of them will have identical rows in the attention matrix. In other words, there can only be at most M unique rows in the attention matrix, one for each centroid in $\mathcal{C}^Q$ (a.k.a pigeonhole principle). You can optimize this further by observing that the attention is just a weighted average over the interaction between centroids in $\mathcal{C}^Q$ and $\mathcal{C}^K$, where the weights are frequency counts of the centroid assignment. This way, you can compute only M^2 dot-products, and factor in the frequency counts wherever needed to ensure that the calculations remain equivalent.

Algorithm: Efficient Computation of O_c

Input: $Q, K, V \in \mathbb{R}^{N \times d}$, centroids $C_Q, C_K \in \mathbb{R}^{M \times d}$

- i. **Cluster Assignment:** Assign each q_i and k_j :

$$a_i = \arg \min_m \|q_i - C_Q[m]\|, \quad b_j = \arg \min_m \|k_j - C_K[m]\|$$

- ii. **Cluster Statistics:** For each key cluster m :

$$n_m = |\{j : b_j = m\}|, \quad S_m = \sum_{j: b_j = m} v_j$$

- iii. **Centroid Attention:** Compute for all $p, m \in [1, M]$:

$$A_{p,m} = \exp(C_Q[p] \cdot C_K[m])$$

- iv. **Normalization:**

$$Z_p = \sum_{m=1}^M A_{p,m} \cdot n_m$$

- v. **Output per Cluster:**

$$O_p = \frac{1}{Z_p} \sum_{m=1}^M A_{p,m} \cdot S_m$$

- vi. **Map Back:**

$$O_i = O_{a_i}$$

Complexity: $O(M^2d + NMd)$

Rubric (3 Marks)

- **1 Mark: Clustering Idea** Correctly groups Q, K into M clusters using C_Q, C_K and notes identical attention rows.
- **1 Mark: Reduced Computation** Computes attention using centroid dot-products only (M^2 operations).
- **1 Mark: Weighting & Mapping** Uses cluster counts/sums (n_m, S_m) and maps outputs back to tokens.

(c) So far we have assumed that $\mathcal{C}$ is available to us. In reality, these centroids have to be updated during training. The way we do it is, at regular intervals of training, we freeze the entire model, do a forward pass over the training set, capture all the $\mathbf{q}_i$ and $\mathbf{k}_j$ vectors and then perform L steps of the k-means algorithm, thereby updating the query and key centroids ($\mathcal{C}^Q$ and $\mathcal{C}^K$). Now, consider a specific case where these L iterations are applied to the query and key representations of a single data instance $\mathbf{X} \in \mathbb{R}^{N \times d}$. Let $\mathbf{S}_{ij} = \frac{\mathbf{q}_i^T \mathbf{k}_j}{\sqrt{d}}$ denote the ground truth attention score, and let $\hat{\mathbf{S}}_{ij} = \frac{(\Phi^Q(\mathbf{q}_i))^T \Phi^K(\mathbf{k}_j)}{\sqrt{d}}$ represent the corresponding clustered approximation between the i^{th} and j^{th} position of the input sequence $\mathbf{X}$. Derive an upper bound for the total aggregate error $\mathcal{L}$, defined as $\mathcal{L} = \sum_{i=1}^N \sum_{j=1}^N |\mathbf{S}_{ij} - \hat{\mathbf{S}}_{ij}|$. Subsequently, prove that this bound is non-increasing throughout the L steps of k-means, thereby demonstrating that the worst-case approximation error is guaranteed to reduce or stay the same with every iteration of k-means. **Useful Inequalities:**

- Triangular Inequality: $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$, where $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ and $\|\cdot\|$ denotes any general norm.
- Cauchy-Schwarz Inequality: $|\langle \mathbf{u}, \mathbf{v} \rangle| \leq \|\mathbf{u}\| \|\mathbf{v}\|$, where $\langle \cdot, \cdot \rangle$ denotes the dot-product.

..4 Let $e_{q,i} = \mathbf{q}_i - c_{q_i}^Q$ and $e_{k,j} = \mathbf{k}_j - c_{k_j}^K$ denote the residual error vectors for the query and key, respectively, where $c_{q_i}^Q \in \mathcal{C}^Q$ and $c_{k_j}^K \in \mathcal{C}^K$ are the centroids assigned for q_i and k_j . Then, we can write:

$$|S_{ij} - \hat{S}_{ij}| = \frac{1}{\sqrt{d}} |q_i^T k_j - (q_i - e_{q,i})^T (k_j - e_{k,j})|$$

Expanding the product:

$$\begin{aligned} |S_{ij} - \hat{S}_{ij}| &= \frac{1}{\sqrt{d}} |q_i^T k_j - (q_i^T k_j - q_i^T e_{k,j} - e_{q,i}^T k_j + e_{q,i}^T e_{k,j})| \\ |S_{ij} - \hat{S}_{ij}| &= \frac{1}{\sqrt{d}} |q_i^T e_{k,j} + e_{q,i}^T k_j - e_{q,i}^T e_{k,j}| \end{aligned}$$

Applying Triangular and Cauchy-Schwarz inequalities, we get:

$$|S_{ij} - \hat{S}_{ij}| \leq \frac{1}{\sqrt{d}} (\|q_i\| \|e_{k,j}\| + \|e_{q,i}\| \|k_j\| + \|e_{q,i}\| \|e_{k,j}\|)$$

Summing over all i, j , we get the aggregate error $\mathcal{L}$:

$$\mathcal{L} \leq \frac{1}{\sqrt{d}} \sum_{i=1}^N \sum_{j=1}^N (\|q_i\| \|e_{k,j}\| + \|e_{q,i}\| \|k_j\| + \|e_{q,i}\| \|e_{k,j}\|)$$

Furthermore, we can distribute the double summation across the three terms to get:

$$L \leq \frac{1}{\sqrt{d}} \left(\sum_{i=1}^N \|q_i\| \sum_{j=1}^N \|e_{k,j}\| + \sum_{i=1}^N \|e_{q,i}\| \sum_{j=1}^N \|k_j\| + \sum_{i=1}^N \|e_{q,i}\| \sum_{j=1}^N \|e_{k,j}\| \right) \quad (6)$$

Now, let us define the k-means objective for both the query and the key representation space as:

$$J^Q = \sum_{i=1}^N \|q_i - c_{q_i}^Q\|^2 = \sum_{i=1}^N \|e_{q,i}\|^2 \quad J^K = \sum_{j=1}^N \|k_j - c_{k_j}^K\|^2 = \sum_{j=1}^N \|e_{k,j}\|^2$$

In Cauchy-Schwarz, if you make $\mathbf{u}$ as a vector of all ones, you get:

$$\sum_{i=1}^d |v_i| \leq \sqrt{d} \sqrt{\sum_{i=1}^d |v_i|^2}$$

Using this, we can write:

$$\sum_{i=1}^N \|e_{q,i}\| \leq \sqrt{N} \sqrt{\sum_{i=1}^N \|e_{q,i}\|^2} \implies \sum_{i=1}^N \|e_{q,i}\| \leq \sqrt{N} \sqrt{J^Q}$$

Using these inequalities, we can reduce eq. (6) to:

$$\mathcal{L} \leq \frac{\sqrt{N}}{\sqrt{d}} \left(A\sqrt{J^K} + B\sqrt{J^Q} + \sqrt{N}\sqrt{J^Q J^K} \right)$$

where $A = \sum_{i=1}^N \|q_i\|$ and $B = \sum_{j=1}^N \|k_j\|$.

Proof of monotonicity : Observe that in our final derived upper bound of $\mathcal{L}$, the terms A, B, N , and d are all constants w.r.t the K-means update. Hence, the behavior of $\mathcal{L}$ is governed exclusively by the K-means objectives, $\sqrt{J^Q}$ and $\sqrt{J^K}$. From the convergence property of K-means, we know that the objectives J^Q and J^K are guaranteed to be non-increasing with every iteration $t \leq L$. Because the bound for $\mathcal{L}$ is directly proportional to these terms, we can conclude that $\mathcal{L}$ is guaranteed to be non-increasing throughout the L steps of the K-means algorithm.

Rubric (4 Marks)

- **0.5 Marks:** For correctly simplifying the $|S_{ij} - \hat{S}_{ij}|$ term.
- **1 Mark:** For using appropriate inequalities (e.g., triangle inequality, Cauchy-Schwarz) to reduce and bound the expression.
- **1 Mark:** For correctly deriving and presenting the final bound expression.
- **1.5 Marks:** For proving monotonicity of the bound and clearly connecting it to the convergence behavior of k-means.

Total: 52
